

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: KAN-2-finalizar-crud-usuario

compare: feature/KAN-3-crud-pet

Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#)

Create pull request

4 commits

24 files changed

1 contributor

Commits on Oct 8, 2025

build(KAN-3): Configura ambiente de teste em memória mgbmoura committed 3 days ago	1795696	<>
feat(KAN-3): Implementa CRUD de Pet, User e Address mgbmoura committed 3 days ago	ce79708	<>
fix(KAN-3): Remove senhas do código e usa variáveis de ambiente mgbmoura committed 3 days ago	a8daf1d	<>
chore: Adiciona regra para ignorar arquivos .md mgbmoura committed 3 days ago	85bbe50	<>

Showing 24 changed files with 788 additions and 150 deletions.

SplitUnified

38 .gitignore

@@ -1,37 +1 @@
1 - HELP.md
2 - target/
3 - .mvn/wrapper/maven-wrapper.jar
4 - !**/src/main/**/target/
5 - !**/src/test/**/target/
6 -
7 - ### STS ###
8 - .apt_generated
9 - .classpath
10 - .factorypath
11 - .project
12 - .settings
13 - .springBeans
14 - .sts4-cache
15 -
16 - ### IntelliJ IDEA ###
17 - .idea
18 - *.iws
19 - *.iml
20 - *.ipr
21 -
22 - ### NetBeans ###

1 + *.md

```
23 - /nbproject/private/
24 - /nbbuild/
25 - /dist/
26 - /nbdist/
27 - /.nb-gradle/
28 - build/
29 - !**/src/main/**/build/
30 - !**/src/test/**/build/
31 -
32 - ### VS Code ###
33 - .vscode/
34 -
35 -
36 - ### Ambiente IDX ###
37 - .idx/
```



16 docker-compose.yml

```
5 db:
6   image: postgres:15-alpine
7   restart: always
8   - environment:
9     - POSTGRES_DB: pet_connect
10
11   - POSTGRES_USER: postgres
12   - POSTGRES_PASSWORD: postgres
13   ports:
14     - "5432:5432"
15   volumes:
16     - postgres_data:/var/lib/postgresql/data
17
18   - "8080:8080"
19   depends_on:
20     - db
21   environment:
22     - SPRING_DATASOURCE_URL:
23       jdbc:postgresql://db:5432/pet_connect
24     - SPRING_DATASOURCE_USERNAME: postgres
25     - SPRING_DATASOURCE_PASSWORD: postgres
26
27   volumes:
28     - postgres_data:
```



```
5 db:
6   image: postgres:15-alpine
7   restart: always
8   + env_file:
9   +   - ./.env # Aponta para o arquivo que conterá
      nossas senhas
10
11   + ports:
12     - "5433:5432"
13   volumes:
14     - postgres_data:/var/lib/postgresql/data
15
16   - "8080:8080"
17   depends_on:
18     - db
19   + env_file:
20   +   - ./.env # Aponta para o mesmo arquivo de
      senhas
21
22   volumes:
23   + postgres_data:
```

6 pom.xml

```
64 <artifactId>spring-boot-starter-
test</artifactId>
65 <scope>test</scope>
66 </dependency>
67 - <!-- <dependency>
68   <groupId>com.h2database</groupId>
69   <artifactId>h2</artifactId>
70   <scope>runtime</scope>
71 - </dependency> -->
72 </dependencies>
73
74 <build>
```

```
64 <artifactId>spring-boot-starter-
test</artifactId>
65 <scope>test</scope>
66 </dependency>
67 + <dependency>
68   <groupId>com.h2database</groupId>
69   <artifactId>h2</artifactId>
70 + <scope>test</scope>
71 + </dependency>
72 </dependencies>
73
74 <build>
```

8 src/main/java/com/petconnectbe/controllers/AddressController.java

```
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
```

```
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.web.bind.annotation.*;
10 +
11 import java.util.List;
```

```

11 import java.util.UUID;
12
13 @RestController
14 @RequestMapping("/addresses")
15 @RequiredArgsConstructor
16 - @CrossOrigin(origins = "*", maxAge = 3600)
17
18 public class AddressController {
19
20     private final AddressService addressService;
21
22     @GetMapping("/{id}")
23     public ResponseEntity<AddressDto>
24     findById(@PathVariable UUID id){
25 -         AddressDto addressDto =
26         addressService.findById(id);
27 -         return ResponseEntity.ok(addressDto);
28
29     }
30 }

```



```

12 import java.util.UUID;
13
14 @RestController
15 @RequestMapping("/addresses")
16 @RequiredArgsConstructor
17 + @CrossOrigin(origins = "http://localhost:3000",
18     maxAge = 3600)
19
20 public class AddressController {
21
22     private final AddressService addressService;
23
24     @GetMapping("/{id}")
25     public ResponseEntity<AddressDto>
26     findById(@PathVariable UUID id){
27 +         return addressService.findById(id)
28 +             .map(ResponseEntity::ok)
29 +             .orElse(ResponseEntity.notFound().build());
30
31     }
32 }

```



63 src/main/java/com/petconnectbe/controllers/PetController.java

... @@ -0,0 +1,63 @@

```

1 + package com.petconnectbe.controllers;
2 +
3 + import
4     com.fasterxml.jackson.databind.ObjectMapper;
5 + import com.petconnectbe.dto.PetDto;
6 + import com.petconnectbe.services.PetService;
7 + import org.springframework.http.HttpStatus;
8 + import org.springframework.http.ResponseEntity;
9 + import org.springframework.web.bind.annotation.*;
10 + import
11     org.springframework.web.multipart.MultipartFile;
12 +
13 + /**
14 +  * Controlador REST para gerenciar as operações
15 +  * relacionadas a Pets.
16 +  * Expõe endpoints para criar, ler, atualizar e
17 +  * deletar informações sobre os animais de estimação.
18 +  */
19 + @RestController
20 + @RequestMapping("/pets")
21 + public class PetController {
22 +
23 +     private final PetService petService;
24 +     private final ObjectMapper objectMapper; //
25 +     Usado para desserializar o JSON do pet
26 +
27 +     /**
28 +      * Construtor para injeção de dependências do
29 +      * PetService e ObjectMapper.
30 +      *
31 +      * @param petService O serviço para lógica de
32 +      * negócios de Pet.
33 +      * @param objectMapper O mapeador de objetos
34 +      * para manipulação de JSON.
35 +      */
36 +     public PetController(PetService petService,
37         ObjectMapper objectMapper) {
38 +         this.petService = petService;
39 +         this.objectMapper = objectMapper;
40 +     }

```

```

32 +
33 +     /**
34 +     * Endpoint para criar um novo Pet.
35 +     * Recebe os dados do pet como uma string JSON
36 +     * e uma imagem (opcional) como multipart/form-data.
37 +     *
38 +     * @param petJson String JSON representando o
39 +     * objeto PetDto.
40 +     * @param image Arquivo de imagem do pet
41 +     * (MultipartFile), pode ser nulo.
42 +     * @return ResponseEntity contendo o PetDto
43 +     * criado e o status HTTP 201 (Created).
44 +     *
45 +     * Retorna HTTP 400 (Bad Request) se a
46 +     * string JSON for malformada.
47 +     */
48 +     @PostMapping(consumes = {"multipart/form-
49 +     data"})
50 +     public ResponseEntity<PetDto> createPet(
51 +         @RequestPart("pet") String petJson,
52 +         @RequestPart(value = "image", required
53 +             = false) MultipartFile image) {
54 +
55 +         try {
56 +             // Converte a string JSON para o
57 +             // objeto PetDto usando o ObjectMapper
58 +             PetDto petDto =
59 +                 objectMapper.readValue(petJson, PetDto.class);
60 +
61 +             // Delega a criação do pet para a
62 +             // camada de serviço
63 +             PetDto createdPet =
64 +                 petService.createPet(petDto, image);
65 +
66 +             // Retorna o objeto criado e o status
67 +             // HTTP 201
68 +             return new ResponseEntity<>
69 +                 (createdPet, HttpStatus.CREATED);
70 +         } catch (Exception e) {
71 +             // Logar a exceção é crucial para a
72 +             // depuração
73 +             e.printStackTrace();
74 +             // Retorna uma resposta de erro
75 +             // genérica para o cliente
76 +             return new ResponseEntity<>
77 +                 (HttpStatus.BAD_REQUEST);
78 +         }
79 +     }
80 + }

```

7 src/main/java/com/petconnectbe/controllers/UserController.java

```

14 @RestController
15 @RequestMapping("/users")
16 @RequiredArgsConstructor
17 - @CrossOrigin(origins = "*", maxAge = 3600)
18
19 public class UserController {
20
21     private final UserService userService;
22
23     @GetMapping("/{id}")
24     public ResponseEntity<UserDto>
25         findById(@PathVariable UUID id) {
26
27         - UserDto userDto =
28         -         userService.findById(id);
29
30         - return ResponseEntity.ok(userDto);

```

```

14 @RestController
15 @RequestMapping("/users")
16 @RequiredArgsConstructor
17 + @CrossOrigin(origins = "http://localhost:3000",
18 +             maxAge = 3600)
19
20 public class UserController {
21
22     private final UserService userService;
23
24     @GetMapping("/{id}")
25     public ResponseEntity<UserDto>
26         findById(@PathVariable UUID id) {
27
28         + return userService.findById(id)
29
30         +         .map(ResponseEntity::ok)

```

		32	+
32	}	33	.orElse(ResponseEntity.notFound().build());
33		34	}
34	@GetMapping()	35	@GetMapping()

1 src/main/java/com/petconnectbe/dto/AddressDto.java

16	private UUID id;	16	private UUID id;
17		17	
18	@NotBlank(message = "O CEP é obrigatório.")	18	@NotBlank(message = "O CEP é obrigatório.")
		19	+ @Size(min = 8, max = 8, message = "O CEP deve conter 8 dígitos.")
19	private String cep;	20	private String cep;
20		21	
21	@NotBlank(message = "A Rua é obrigatório.")	22	@NotBlank(message = "A Rua é obrigatório.")

72 src/main/java/com/petconnectbe/dto/PetDto.java

...	@@ -0,0 +1,72 @@	1	+ package com.petconnectbe.dto;
		2	+
		3	+ import lombok.Data;
		4	+ import lombok.NoArgsConstructor;
		5	+
		6	+ import java.util.UUID;
		7	+
		8	+ /**
		9	+ * DTO (Data Transfer Object) para dados de Pet.
		10	+ * Usado para transferir informações de pets entre a camada de controle e os clientes (ex: frontend).
		11	+ * O Lombok @Data gera automaticamente getters, setters, toString, equals e hashCode.
		12	+ * O Lombok @NoArgsConstructor gera um construtor sem argumentos.
		13	+ */
		14	+ @Data
		15	+ @NoArgsConstructor
		16	+ public class PetDto {
		17	+
		18	+ /**
		19	+ * O identificador único do pet. Gerado pelo banco de dados.
		20	+ */
		21	+ private Integer id;
		22	+
		23	+ /**
		24	+ * O ID do tutor (usuário) associado a este pet.
		25	+ */
		26	+ private UUID tutorId;
		27	+
		28	+ /**
		29	+ * O nome do pet.
		30	+ */
		31	+ private String name;
		32	+
		33	+ /**
		34	+ * A espécie do pet (ex: Cachorro, Gato).
		35	+ */
		36	+ private String species;
		37	+
		38	+ /**
		39	+ * A raça ou raças do pet.
		40	+ */
		41	+ private String races;

```

42 +
43 + /**
44 +  * O peso do pet em quilogramas.
45 +  */
46 + private Double weight;
47 +
48 + /**
49 +  * A idade do pet em anos.
50 +  */
51 + private Integer age;
52 +
53 + /**
54 +  * O sexo do pet (ex: Macho, Fêmea).
55 +  */
56 + private String sex;
57 +
58 + /**
59 +  * O número do microchip de identificação, se
    houver.
60 +  */
61 + private String microchipNumber;
62 +
63 + /**
64 +  * Um campo de texto livre para informações
    adicionais sobre o pet.
65 +  */
66 + private String about;
67 +
68 + /**
69 +  * A URL da imagem do pet. Este campo é
    populado pelo backend após o upload.
70 +  */
71 + private String imageUrl;
72 + }

```

36 src/main/java/com/petconnectbe/models/Address.java

```

5 import jakarta.persistence.Column;
6 import jakarta.persistence.Entity;
7 import jakarta.persistence.GeneratedValue;

8 import jakarta.persistence.Id;
9 import jakarta.persistence.Table;
10 import lombok.Data;

11
12 import java.io.Serializable;
13 import java.util.UUID;
14

```

```

15 @Entity
16 @Table(name = "address_tb")
17 @Data
18 @JsonInclude(JsonInclude.Include.NON_NULL)
19 @JsonIgnoreProperties(ignoreUnknown = true)
20 public class Address implements Serializable {
21

```

22 @Id

```

5 import jakarta.persistence.Column;
6 import jakarta.persistence.Entity;
7 import jakarta.persistence.GeneratedValue;
8 + import jakarta.persistence.GenerationType;
9 import jakarta.persistence.Id;
10 import jakarta.persistence.Table;
11 import lombok.Data;

12
13 import java.io.Serializable;
14 import java.util.UUID;
15
16 + /**
17 +  * Entidade JPA que representa um Endereço no
    banco de dados.
18 +  * Mapeada para a tabela "address_tb".
19 +  * Implementa Serializable para suportar
    mecanismos de serialização do Java.
20 +  */
21 @Entity
22 @Table(name = "address_tb")
23 @Data
24 @JsonInclude(JsonInclude.Include.NON_NULL)
25 @JsonIgnoreProperties(ignoreUnknown = true)
26 public class Address implements Serializable {
27
28 + /**
29 +  * Chave primária da entidade Endereço.
30 +  * É um UUID gerado automaticamente.
31 +  */
32 @Id

```

```

23     @Column(name = "address_id")
24 -   @GeneratedValue(generator = "UUID")
25 -   private UUID addressId;
26
27     @Column(name = "cep", nullable = false, length
28 = 9)
29     private String cep;
30
31     @Column(name = "street", nullable = false,
32 length = 100)
33     private String street;
34
35     @Column(name = "neighborhood", nullable =
36 false, length = 100)
37     private String neighborhood;
38
39     @Column(name = "city", nullable = false,
40 length = 100)
41     private String city;
42
43     @Column(name = "uf", nullable = false, length
44 = 2)
45     private String uf;
46
47     -
48     -
49     }

```

```

33 +   @GeneratedValue(strategy =
34 GenerationType.AUTO)
35     @Column(name = "address_id")
36     private UUID id;
37
38     /**
39      * Código de Endereçamento Postal (CEP).
40      * Armazenado como uma string de até 9
41      * caracteres. Não pode ser nulo.
42      */
43     @Column(name = "cep", nullable = false, length
44 = 9)
45     private String cep;
46
47     /**
48      * Nome da rua, avenida ou logradouro.
49      * Não pode ser nulo.
50      */
51     @Column(name = "street", nullable = false,
52 length = 100)
53     private String street;
54
55     /**
56      * Nome do bairro.
57      * Não pode ser nulo.
58      */
59     @Column(name = "neighborhood", nullable =
60 false, length = 100)
61     private String neighborhood;
62
63     /**
64      * Nome da cidade.
65      * Não pode ser nulo.
66      */
67     @Column(name = "city", nullable = false,
68 length = 100)
69     private String city;
70
71     /**
72      * Sigla da Unidade Federativa (Estado), com 2
73      * caracteres.
74      * Ex: "SP", "RJ". Não pode ser nulo.
75      */
76     @Column(name = "uf", nullable = false, length
77 = 2)
78     private String uf;
79
80     }

```

84  src/main/java/com/petconnectbe/models/Pet.java 

... @@ -0,0 +1,84 @@

```

1 + package com.petconnectbe.models;
2 +
3 + import jakarta.persistence.*;
4 + import lombok.Data;
5 + import lombok.NoArgsConstructor;
6 +
7 + /**
8 +  * Entidade JPA que representa um Pet no banco de
9 +  * dados.
10 +  * Mapeada para a tabela "pets". Contém todas as
11 +  * informações
12 +  * relevantes sobre o animal de estimação.

```

```

11 + */
12 + @Data
13 + @NoArgsConstructor
14 + @Entity
15 + @Table(name = "pets")
16 + public class Pet {
17 +
18 +     /**
19 +      * Chave primária da entidade Pet.
20 +      * O valor é gerado automaticamente pelo banco
      de dados (estratégia de identidade).
21 +     */
22 +     @Id
23 +     @GeneratedValue(strategy =
      GenerationType.IDENTITY)
24 +     private Integer id;
25 +
26 +     /**
27 +      * Nome do pet. Campo obrigatório.
28 +     */
29 +     @Column(nullable = false)
30 +     private String name;
31 +
32 +     /**
33 +      * Espécie do pet (ex: "Cachorro", "Gato",
      "Coelho").
34 +     */
35 +     private String species;
36 +
37 +     /**
38 +      * Raça do pet.
39 +     */
40 +     private String races;
41 +
42 +     /**
43 +      * Peso do pet, geralmente em quilogramas.
44 +     */
45 +     private Double weight;
46 +
47 +     /**
48 +      * Idade do pet em anos.
49 +     */
50 +     private Integer age;
51 +
52 +     /**
53 +      * Sexo do pet (ex: "Macho", "Fêmea").
54 +     */
55 +     private String sex;
56 +
57 +     /**
58 +      * Número de identificação do microchip do
      pet, caso possua.
59 +     */
60 +     private String microchipNumber;
61 +
62 +     /**
63 +      * Campo de texto livre para informações
      adicionais sobre o pet,
64 +      * como comportamento, histórico médico, etc.
65 +      * Mapeado para um tipo de coluna que suporta
      textos longos.
66 +     */
67 +     @Column(columnDefinition = "TEXT")
68 +     private String about;
69 +
70 +     /**

```

```

71 +     * URL da imagem de perfil do pet. Pode ser um
    link para um serviço de
72 +     * armazenamento de arquivos ou um caminho
    local servido pela aplicação.
73 +     */
74 +     private String imageUrl;
75 +
76 +     /**
77 +     * Relacionamento Muitos-para-Um com a
    entidade User, representando o tutor do pet.
78 +     * - `fetch = FetchType.LAZY`: O tutor não é
    carregado do banco de dados até que seja
    explicitamente acessado.
79 +     * - `nullable = false`: Todo pet deve
    obrigatoriamente ter um tutor associado.
80 +     */
81 +     @ManyToOne(fetch = FetchType.LAZY)
82 +     @JoinColumn(name = "tutor_id", nullable =
    false)
83 +     private User tutor;
84 + }

```

66 src/main/java/com/petconnectbe/models/User.java

```

2
3 import
    com.fasterxml.jackson.annotation.JsonIgnorePropert
    ies;
4 import
    com.fasterxml.jackson.annotation.JsonInclude;
5 - import jakarta.persistence.CascadeType;
6 - import jakarta.persistence.Column;
7 - import jakarta.persistence.Entity;
8 - import jakarta.persistence.GeneratedValue;
9 - import jakarta.persistence.Id;
10 - import jakarta.persistence.JoinColumn;
11 - import jakarta.persistence.OneToOne;
12 - import jakarta.persistence.Table;
13 import lombok.Data;
14
15 import java.io.Serializable;
16 import java.time.LocalDate;
17 import java.util.UUID;
18
19 @Entity
20 @Data
21 @Table(name = "user_tb")
22 @JsonInclude(JsonInclude.Include.NON_NULL)
23 @JsonIgnoreProperties(ignoreUnknown = true)
24 public class User implements Serializable {
25
26     @Id
27 -     @GeneratedValue(generator = "UUID")
28
29     @Column(name = "user_id")
30 -     private UUID userId;

```

```

2
3 import
    com.fasterxml.jackson.annotation.JsonIgnorePropert
    ies;
4 import
    com.fasterxml.jackson.annotation.JsonInclude;
5 + import jakarta.persistence.*;
6
7 import lombok.Data;
8
9 import java.io.Serializable;
10 import java.time.LocalDate;
11 import java.util.UUID;
12
13 + /**
14 + * Entidade JPA que representa um Usuário no banco
    de dados.
15 + * Pode ser tanto uma pessoa física (tutor) quanto
    uma pessoa jurídica (abrigo).
16 + * Mapeada para a tabela "user_tb".
17 + */
18 @Entity
19 @Data
20 @Table(name = "user_tb")
21 @JsonInclude(JsonInclude.Include.NON_NULL)
22 @JsonIgnoreProperties(ignoreUnknown = true)
23 public class User implements Serializable {
24
25 +     /**
26 +     * Chave primária da entidade Usuário.
27 +     * É um UUID gerado automaticamente.
28 +     */
29 +     @Id
30 +     @GeneratedValue(strategy =
    GenerationType.AUTO)
31 +     @Column(name = "user_id")
32 +     private UUID id;

```

```

30
31     @Column(name = "type", nullable = false,
32         length = 30)
33     private String type;
34 -     @Column(name = "name", nullable = false, length
35         = 150)
36
37     private String name;
38
39 -     @Column(name = "email", nullable = false,
40         length = 254)
41
42     private String email;
43
44     @Column(name = "phone", nullable = false,
45         length = 20)
46     private String phone;
47
48
49
50
51
52
53
54
55     @Column(name = "birth_foundation_date",
56         nullable = false)
57     private LocalDate birthOrFoundationDate;
58
59 -     @Column(name = "cpf_cnpj", nullable = false,
60         length = 18)
61
62     private String cpfOrCnpj;
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79

```

```

32
33 +     /**
34 +      * Tipo de usuário (ex: "TUTOR", "ABRIGO").
35 +      * Campo obrigatório para diferenciar os tipos
36 +      * de conta.
37 +      */
38     @Column(name = "type", nullable = false,
39         length = 30)
40     private String type;
41
42 +     /**
43 +      * Nome completo do usuário ou razão social do
44 +      * abrigo.
45 +      * Campo obrigatório.
46 +      */
47     @Column(name = "name", nullable = false,
48         length = 150)
49     private String name;
50
51 +     /**
52 +      * Endereço de e-mail do usuário.
53 +      * Utilizado para login e comunicação. Deve
54 +      * ser único. Campo obrigatório.
55 +      */
56     @Column(name = "email", nullable = false,
57         unique = true, length = 254)
58     private String email;
59
60 +     /**
61 +      * Número de telefone para contato.
62 +      * Campo obrigatório.
63 +      */
64     @Column(name = "phone", nullable = false,
65         length = 20)
66     private String phone;
67
68 +     /**
69 +      * Data de nascimento (para pessoa física) ou
70 +      * data de fundação (para pessoa jurídica).
71 +      * Campo obrigatório.
72 +      */
73     @Column(name = "birth_foundation_date",
74         nullable = false)
75     private LocalDate birthOrFoundationDate;
76
77 +     /**
78 +      * Cadastro de Pessoa Física (CPF) ou Cadastro
79 +      * Nacional da Pessoa Jurídica (CNPJ).
80 +      * O formato esperado é "xxx.xxx.xxx-xx" para
81 +      * CPF e "xx.xxx.xxx/xxxx-xx" para CNPJ.
82 +      * Deve ser único. Campo obrigatório.
83 +      */
84     @Column(name = "cpf_cnpj", nullable = false,
85         unique = true, length = 18)
86     private String cpfOrCnpj;
87
88 +     /**
89 +      * Relacionamento Um-para-Um com a entidade
90 +      * Address.
91 +      * - `cascade = CascadeType.ALL`: Operações de
92 +      * persistência, atualização e remoção
93 +      * no usuário serão cascateadas para o
94 +      * endereço associado.
95 +      */
96

```

```

49     @OneToOne(cascade = CascadeType.ALL)
50 -   @JoinColumn(name = "adress_id")
51     private Address address;
52
53     @Column(name = "password", length = 100)
54     private String password;
55 -
56 -
57 }

```

```

80 +     * - `@JoinColumn`: Especifica a chave
    estrangeira na tabela de usuários.
81 +     */
82     @OneToOne(cascade = CascadeType.ALL)
83 +   @JoinColumn(name = "address_id")
84     private Address address;
85
86 +     /**
87 +     * Senha do usuário, armazenada de forma
    criptografada (hash).
88 +     */
89     @Column(name = "password", length = 100)
90     private String password;
91 }

```

18 src/main/java/com/petconnectbe/repositories/AddressRepository.java

```

6     import java.util.Optional;
7     import java.util.UUID;
8
9     public interface AddressRepository extends
    JpaRepository<Address, UUID> {
10 -     Optional<Address> findEnderecoByUf(String
    estadoUF);
11 }

```

```

6     import java.util.Optional;
7     import java.util.UUID;
8
9 +     /**
10 +     * Repositório Spring Data JPA para a entidade
    {@link Address}.
11 +     * <p>
12 +     * Fornece métodos CRUD (Create, Read, Update,
    Delete) para a entidade Address,
13 +     * além de permitir a definição de métodos de
    consulta personalizados.
14 +     */
15     public interface AddressRepository extends
    JpaRepository<Address, UUID> {
16 +
17 +     /**
18 +     * Busca o primeiro endereço encontrado
    correspondente a uma sigla de Unidade Federativa
    (UF).
19 +     * <p>
20 +     * Nota: Este método retornará apenas um
    resultado, mesmo que múltiplos endereços
21 +     * compartilhem a mesma UF.
22 +     *
23 +     * @param uf A sigla do estado (UF) para a
    busca. Ex: "SP", "RJ".
24 +     * @return Um {@link Optional} contendo o
    {@link Address} encontrado, ou {@link
    Optional#empty()} se nenhum for encontrado.
25 +     */
26 +     Optional<Address> findByUf(String uf);
27 }

```

14 src/main/java/com/petconnectbe/repositories/PetRepository.java

```
...    @@ -0,0 +1,14 @@
```

```

1 + package com.petconnectbe.repositories;
2 +
3 + import com.petconnectbe.models.Pet;
4 + import
    org.springframework.data.jpa.repository.JpaReposit
    ory;
5 +
6 + /**
7 + * Repositório Spring Data JPA para a entidade
    {@link Pet}.

```

```

8 + * <p>
9 + * Fornece métodos CRUD (Create, Read, Update,
    Delete) para a entidade Pet
10 + * sem a necessidade de implementação explícita,
    além de permitir a definição de
11 + * métodos de consulta personalizados que o Spring
    Data JPA implementará automaticamente.
12 + */
13 + public interface PetRepository extends
    JpaRepository<Pet, Integer> {
14 + }

```

18 src/main/java/com/petconnectbe/repositories/UserRepository.java

```

6 import java.util.Optional;
7 import java.util.UUID;
8
9 public interface UserRepository extends
    JpaRepository<User, UUID> {
10
11 -
12
13 -
14 -
15 }

```

```

6 import java.util.Optional;
7 import java.util.UUID;
8
9 + /**
10 + * Repositório Spring Data JPA para a entidade
    {@link User}.
11 + * <p>
12 + * Fornece métodos CRUD (Create, Read, Update,
    Delete) para a entidade User,
13 + * além de permitir a definição de métodos de
    consulta personalizados.
14 + */
15 public interface UserRepository extends
    JpaRepository<User, UUID> {
16
17 + /**
18 + * Busca um usuário pelo seu endereço de e-
    mail.
19 + * <p>
20 + * Como o campo de e-mail é definido como
    único na entidade {@link User},
21 + * este método retornará no máximo um usuário.
22 + *
23 + * @param email O e-mail a ser pesquisado.
24 + * @return Um {@link Optional} contendo o
    {@link User} encontrado, ou {@link
    Optional#empty()} se nenhum usuário com o e-mail
    fornecido for encontrado.
25 + */
26 Optional<User> findByEmail(String email);
27 }

```

47 src/main/java/com/petconnectbe/services/AddressService.java

```

4 import com.petconnectbe.models.Address;
5
6 import java.util.List;
7
8 import java.util.UUID;
9
10 public interface AddressService {

```

```

4 import com.petconnectbe.models.Address;
5
6 import java.util.List;
7 + import java.util.Optional;
8 import java.util.UUID;
9
10 + /**
11 + * Interface que define o contrato para as
    operações de negócio relacionadas a Endereços.
12 + * <p>
13 + * Esta camada de serviço abstrai a lógica de
    negócio e as interações com o repositório
14 + * de dados para a entidade {@link Address}.
15 + */
16 public interface AddressService {
17
18 + /**

```

```

11     AddressDto save(AddressDto dto);
12
13     List<AddressDto> findAll();
14
15     - AddressDto findById(UUID id);
16     -
17     -
18     - AddressDto toDto(Address address);
19     -
20     - Address toEntity(AddressDto addressDto);
21 }

```

```

19 +     * Salva ou atualiza um endereço no banco de
    +     dados.
20 +     *
21 +     * @param dto O DTO contendo as informações do
    +     endereço a ser salvo.
22 +     * @return O {@link AddressDto} representando
    +     o endereço salvo.
23 +     */
24     AddressDto save(AddressDto dto);
25
26 +     /**
27 +     * Retorna uma lista com todos os endereços
    +     cadastrados.
28 +     *
29 +     * @return Uma lista de {@link AddressDto}.
30 +     */
31     List<AddressDto> findAll();
32
33 +     /**
34 +     * Busca um endereço pelo seu identificador
    +     único (ID).
35 +     *
36 +     * @param id O UUID do endereço a ser buscado.
37 +     * @return Um {@link Optional} contendo o
    +     {@link AddressDto} se encontrado, ou {@link
    +     Optional#empty()} caso contrário.
38 +     */
39     Optional<AddressDto> findById(UUID id);
40 +
41 +     /**
42 +     * Converte uma entidade {@link Address} para
    +     seu respectivo {@link AddressDto}.
43 +     *
44 +     * @param address A entidade a ser convertida.
45 +     * @return O DTO correspondente.
46 +     */
47     AddressDto toDto(Address address);
48 +
49 +     /**
50 +     * Converte um {@link AddressDto} para sua
    +     respectiva entidade {@link Address}.
51 +     *
52 +     * @param addressDto O DTO a ser convertido.
53 +     * @return A entidade correspondente.
54 +     */
55     Address toEntity(AddressDto addressDto);
56 }

```

▼ 27  src/main/java/com/petconnectbe/services/FileStorageService.java 

...  -0,0 +1,27 

```

1 + package com.petconnectbe.services;
2 +
3 + import
    + org.springframework.web.multipart.MultipartFile;
4 +
5 + /**
6 + * Define o contrato para o serviço de
    + armazenamento de arquivos, como imagens de pets.
7 + * <p>
8 + * Abstrai a lógica de como e onde os arquivos são
    + fisicamente armazenados (por exemplo,
9 + * em um sistema de arquivos local, em um bucket
    + de nuvem como Amazon S3, etc.),
10 + * expando uma operação de armazenamento simples.
11 + */

```

```

12 + public interface FileStorageService {
13 +
14 +     /**
15 +      * Armazena um arquivo enviado e retorna um
16 +      * identificador de acesso a ele.
17 +      * <p>
18 +      * A implementação deste método deve lidar com
19 +      * a recepção do {@link MultipartFile},
20 +      * salvá-lo em um local de armazenamento
21 +      * persistente e seguro, e retornar uma
22 +      * string (como uma URL ou um nome de arquivo
23 +      * único) que possa ser usada
24 +      * posteriormente para acessar o arquivo.
25 +      *
26 +      * @param file O arquivo a ser armazenado,
27 +      * recebido de um upload de formulário.
28 +      * @return Uma {@code String} representando o
29 +      * caminho ou a URL para o arquivo armazenado.
30 +      * Este valor será usado para associar
31 +      * o arquivo a outras entidades, como um Pet.
32 +      */
33 +     String store(MultipartFile file);
34 + }

```

31 src/main/java/com/petconnectbe/services/PetService.java

... @@ -0,0 +1,31 @@

```

1 + package com.petconnectbe.services;
2 +
3 + import com.petconnectbe.dto.PetDto;
4 + import
5 +     org.springframework.web.multipart.MultipartFile;
6 +
7 + /**
8 +  * Define o contrato para o serviço de
9 +  * gerenciamento de Pets.
10 +  * <p>
11 +  * Esta interface abstrai a lógica de negócio para
12 +  * todas as operações relacionadas a pets,
13 +  * incluindo a criação, recuperação, atualização e
14 +  * exclusão de informações de pets.
15 +  * A implementação deste serviço irá coordenar as
16 +  * interações com a camada de persistência
17 +  * e outros serviços, como o {@link
18 +  * FileStorageService}, para gerenciar os dados
19 +  * completos de um pet.
20 +  */
21 + public interface PetService {
22 +
23 +     /**
24 +      * Cria um novo Pet no sistema, associando uma
25 +      * imagem a ele.
26 +      * <p>
27 +      * O processo de criação envolve:
28 +      * <ol>
29 +      * <li>Validar os dados do {@code petDto}.
30 +      * </li>
31 +      * <li>Se uma {@code image} for fornecida,
32 +      * ela é armazenada usando o {@link
33 +      * FileStorageService}.</li>
34 +      * <li>As informações do pet, incluindo a
35 +      * URL da imagem, são salvas no banco de dados.</li>
36 +      * </ol>
37 +      *
38 +      * @param petDto O objeto de transferência de
39 +      * dados (DTO) contendo as informações do novo pet.

```

```

27 +     * @param image O arquivo de imagem do pet a
    ser enviado. Este parâmetro é opcional.
28 +     * @return O {@link PetDto} representando o
    pet que foi criado e salvo.
29 +     */
30 +     PetDto createPet(PetDto petDto, MultipartFile
    image);
31 + }

```

43 src/main/java/com/petconnectbe/services/UserService.java

```

3 import com.petconnectbe.dto.UserDto;
4
5 import java.util.List;
6
7 import java.util.UUID;

```

```

8 public interface UserService {
9

```

```

10     UserDto save(UserDto userDto);
11
12     - UserDto findById(UUID id);
13     -

```

```

14     List<UserDto> findAll();
15

```

```

3 import com.petconnectbe.dto.UserDto;
4
5 import java.util.List;
6 + import java.util.Optional;
7 import java.util.UUID;
8
9 + /**
10 +  * Define o contrato para o serviço de
    gerenciamento de Usuários.
11 +  * <p>
12 +  * Esta interface abstrai a lógica de negócio para
    todas as operações relacionadas a usuários,
13 +  * como criação, busca, atualização e exclusão.
14 +  * A implementação deste serviço será responsável
    por coordenar as interações com a camada de
15 +  * persistência e garantir a integridade dos dados
    do usuário.
16 +  */
17 public interface UserService {
18
19 +     /**
20 +     * Cria um novo usuário no sistema.
21 +     *
22 +     * @param userDto O DTO contendo as
    informações do novo usuário.
23 +     * @return O {@link UserDto} representando o
    usuário que foi salvo.
24 +     */
25     UserDto save(UserDto userDto);
26
27 +     /**
28 +     * Busca um usuário pelo seu identificador
    único (ID).
29 +     *
30 +     * @param id O UUID do usuário a ser buscado.
31 +     * @return Um {@link Optional} contendo o
    {@link UserDto} se o usuário for encontrado,
32 +     * ou {@link Optional#empty()} caso
    contrário.
33 +     */
34 +     Optional<UserDto> findById(UUID id);
35 +
36 +     /**
37 +     * Retorna uma lista com todos os usuários
    cadastrados no sistema.
38 +     *
39 +     * @return Uma lista de {@link UserDto}.
40 +     */
41     List<UserDto> findAll();
42
43 +     /**
44 +     * Atualiza as informações de um usuário
    existente.
45 +     *
46 +     * @param id O UUID do usuário a ser
    atualizado.

```

```

16         UserDto update(UUID id, UserDto userDto);
17
18         void deleteById(UUID id);
19     }
20 }

```

```

47 +         * @param userDto O DTO contendo as novas
    informações do usuário.
48 +         * @return O {@link UserDto} representando o
    usuário com as informações atualizadas.
49 +         */
50         UserDto update(UUID id, UserDto userDto);
51
52 +     /**
53 +      * Exclui um usuário do sistema com base no
    seu ID.
54 +      *
55 +      * @param id O UUID do usuário a ser excluído.
56 +      */
57         void deleteById(UUID id);
58
59     }

```

67 src/main/java/com/petconnectbe/services/impl/AddressServiceImpl.java

```

4     import com.petconnectbe.models.Address;
5     import
    com.petconnectbe.repositories.AddressRepository;
6     import com.petconnectbe.services.AddressService;
7
8     import lombok.RequiredArgsConstructor;
9     import org.springframework.stereotype.Service;
10
11     import java.util.List;
12
13     import java.util.UUID;
14     import java.util.stream.Collectors;
15
16
17
18
19
20

```

```

13     @Service
14     @RequiredArgsConstructor
15     public class AddressServiceImpl implements
    AddressService {
16
17         private final AddressRepository
    addressRepository;
18
19

```

@Override

```

4     import com.petconnectbe.models.Address;
5     import
    com.petconnectbe.repositories.AddressRepository;
6     import com.petconnectbe.services.AddressService;
7 +     import
    jakarta.persistence.EntityNotFoundException;
8     import lombok.RequiredArgsConstructor;
9     import org.springframework.stereotype.Service;
10 +     import
    org.springframework.transaction.annotation.Transactiona
    ctional;
11 +
12     import java.util.List;
13 +     import java.util.Optional;
14     import java.util.UUID;
15     import java.util.stream.Collectors;
16
17 + /**
18 +  * Implementação do serviço {@link
    AddressService} para gerenciar endereços.
19 +  * <p>
20 +  * Esta classe implementa a lógica de negócio
    para operações relacionadas a endereços,
21 +  * utilizando o {@link AddressRepository} para
    interagir com o banco de dados.
22 +  * As transações são gerenciadas pelo Spring.
23 +  */
24     @Service
25     @RequiredArgsConstructor
26     public class AddressServiceImpl implements
    AddressService {
27
28         private final AddressRepository
    addressRepository;
29
30 +     /**
31 +      * {@inheritDoc}
32 +      * <p>
33 +      * Se o DTO fornecido tiver um ID, o método
    tentará atualizar o endereço existente.
34 +      * Caso contrário, um novo endereço será
    criado.
35 +      *
36 +      * @throws EntityNotFoundException se um ID
    for fornecido mas o endereço correspondente não
    for encontrado.
37 +      */
38     @Override

```

```

21     public AddressDto save(AddressDto dto) {
22 -         Address address = new Address();

23         address.setCep(dto.getCep());
24         address.setStreet(dto.getStreet());
25
26         address.setNeighborhood(dto.getNeighborhood());
27         address.setCity(dto.getCity());
28         address.setUf(dto.getUf());
29 -         Address salvo =
30 -         addressRepository.save(address);
31 -         return toDto(salvo);
32     }

33     @Override

34     public List<AddressDto> findAll() {
35         return addressRepository.findAll()
36 -         .stream()
37 -         .map(this::toDto)
38 -         .collect(Collectors.toList());
39     }

40

41     @Override
42 -     public AddressDto findById(UUID id) {
43 -         Address address =
44 -         addressRepository.findById(id)
45 -         .orElseThrow(() -> new
46 -         RuntimeException("Endereço não encontrado."));
47 -         return toDto(address);
48 -     }
49 -

50     @Override
51     public AddressDto toDto(Address address) {

52         AddressDto dto = new AddressDto();

53         dto.setCep(address.getCep());
54         dto.setStreet(address.getStreet());

55         dto.setNeighborhood(address.getNeighborhood());

56         return dto;
57     }

```

```

39 +     @Transactional
40     public AddressDto save(AddressDto dto) {
41 +         Address address;
42 +         if (dto.getId() != null) {
43 +             address =
44 +             addressRepository.findById(dto.getId())
45 +             .orElseThrow(() -> new
46 +             EntityNotFoundException("Endereço não encontrado
47 +             com o ID: " + dto.getId()));
48 +         } else {
49 +             address = new Address();
50 +         }
51
52         address.setCep(dto.getCep());
53         address.setStreet(dto.getStreet());
54
55         address.setNeighborhood(dto.getNeighborhood());
56         address.setCity(dto.getCity());
57         address.setUf(dto.getUf());
58
59 +         Address savedAddress =
60 +         addressRepository.save(address);
61 +         return toDto(savedAddress);
62     }

63     /**
64     * {@inheritDoc}
65     */
66     @Override
67     @Transactional(readOnly = true)
68     public List<AddressDto> findAll() {
69         return addressRepository.findAll()
70         .stream()
71         .map(this::toDto)
72         .collect(Collectors.toList());
73     }

74     /**
75     * {@inheritDoc}
76     */
77     @Override
78     @Transactional(readOnly = true)
79     public Optional<AddressDto> findById(UUID id)
80     {
81         return
82         addressRepository.findById(id).map(this::toDto);
83     }

84     /**
85     * {@inheritDoc}
86     */
87     @Override
88     public AddressDto toDto(Address address) {
89 +         if (address == null) {
90 +             return null;
91 +         }
92
93         AddressDto dto = new AddressDto();
94
95         dto.setId(address.getId());
96         dto.setCep(address.getCep());
97         dto.setStreet(address.getStreet());
98
99         dto.setNeighborhood(address.getNeighborhood());
100
101         return dto;
102     }

103     /**

```

```

61     @Override
62     public Address toEntity(AddressDto dto) {
63         if (dto == null) {
64             return null;
65         }
66         Address address = new Address();
67
68         address.setCep(dto.getCep());
69         address.setStreet(dto.getStreet());
70
71         address.setNeighborhood(dto.getNeighborhood());

```

```

99     + * {@inheritDoc}
100    + */
101    @Override
102    public Address toEntity(AddressDto dto) {
103        if (dto == null) {
104            return null;
105        }
106        Address address = new Address();
107    + address.setId(dto.getId());
108        address.setCep(dto.getCep());
109        address.setStreet(dto.getStreet());
110
111        address.setNeighborhood(dto.getNeighborhood());

```

▼ 44  src/main/java/com/petconnectbe/services/impl/FileStorageServiceImpl.java 

... @@ -0,0 +1,44 @@

```

1  + package com.petconnectbe.services.impl;
2  +
3  + import
4  +     com.petconnectbe.services.FileStorageService;
5  + import
6  +     org.springframework.stereotype.Service;
7  + import
8  +     org.springframework.web.multipart.MultipartFile;
9  + import
10 +     java.util.UUID;
11 +
12 + /**
13 +  * Implementação <strong>simulada</strong> do
14 +  * serviço
15 +  * {@link FileStorageService}.
16 +  * <p>
17 +  * <strong>ATENÇÃO:</strong> Esta é uma
18 +  * implementação para fins de
19 +  * desenvolvimento e teste.
20 +  * Ela não armazena fisicamente os arquivos,
21 +  * apenas simula o processo e retorna
22 +  * um caminho fictício.
23 +  * Em um ambiente de produção, esta classe deve
24 +  * ser substituída por uma
25 +  * implementação real
26 +  * que salve os arquivos em um sistema de
27 +  * armazenamento persistente (ex: Amazon
28 +  * S3, Google Cloud Storage, ou disco local).
29 +  */
30 + @Service
31 + public class FileStorageServiceImpl implements
32 +     FileStorageService {
33 +
34 +     /**
35 +      * {@inheritDoc}
36 +      * <p>
37 +      * Esta implementação simulada gera um nome de
38 +      * arquivo único e imprime o nome do
39 +      * arquivo original no console para fins de
40 +      * depuração. O caminho retornado é um
41 +      * caminho fictício que pode ser usado pelo
42 +      * front-end para exibir imagens de
43 +      * exemplo.
44 +      */
45 +     @Override
46 +     public String store(MultipartFile file) {
47 +         // Simula o log do armazenamento do
48 +         // arquivo
49 +         System.out.println("Simulando o
50 +             armazenamento do arquivo: " +
51 +             file.getOriginalFilename());

```

```

36 +
37 +         // Gera um nome de arquivo único para
           evitar conflitos
38 +         String fileName =
           UUID.randomUUID().toString() + "-" +
           file.getOriginalFilename();
39 +
40 +         // Retorna um caminho fictício que o
           front-end pode usar para exibir uma imagem
41 +         // de exemplo
42 +         return "/static/images/pets/" + fileName;
43 +     }
44 + }

```

▼ 86  src/main/java/com/petconnectbe/services/impl/PetServiceImpl.java 

... @@ -0,0 +1,86 @@

```

1 + package com.petconnectbe.services.impl;
2 +
3 + import com.petconnectbe.dto.PetDto;
4 + import com.petconnectbe.models.Pet;
5 + import com.petconnectbe.models.User;
6 + import
           com.petconnectbe.repositories.PetRepository;
7 + import
           com.petconnectbe.repositories.UserRepository;
8 + import
           com.petconnectbe.services.FileStorageService;
9 + import com.petconnectbe.services.PetService;
10 + import
           jakarta.persistence.EntityNotFoundException;
11 + import lombok.RequiredArgsConstructor;
12 + import org.springframework.beans.BeanUtils;
13 + import org.springframework.stereotype.Service;
14 + import
           org.springframework.transaction.annotation.Transactional;
15 + import
           org.springframework.web.multipart.MultipartFile;
16 +
17 + /**
18 +  * Implementação do serviço {@link PetService}
           para gerenciar os dados dos Pets.
19 +  * <p>
20 +  * Esta classe orquestra as operações de negócio
           relacionadas a pets, como a
21 +  * criação,
22 +  * interagindo com os repositórios e outros
           serviços (como o
23 +  * {@link FileStorageService})
24 +  * para cumprir suas responsabilidades.
25 +  */
26 + @Service
27 + @RequiredArgsConstructor
28 + public class PetServiceImpl implements PetService
           {
29 +
30 +     private final PetRepository petRepository;
31 +     private final UserRepository userRepository;
32 +     private final FileStorageService
           fileStorageService;
33 +
34 +     /**
35 +      * {@inheritDoc}
36 +      *

```

```

37 +      * @throws EntityNotFoundException se o tutor
      (usuário) não for encontrado.
38 +      */
39 +      @Override
40 +      @Transactional
41 +      public PetDto createPet(PetDto petDto,
      MultipartFile image) {
42 +          User tutor =
      userRepository.findById(petDto.getTutorId())
43 +              .orElseThrow(() -> new
      EntityNotFoundException("Tutor não encontrado com
      o ID: " + petDto.getTutorId()));
44 +
45 +          Pet pet = toEntity(petDto);
46 +          pet.setTutor(tutor); // Associa a entidade
      User completa
47 +
48 +          if (image != null && !image.isEmpty()) {
49 +              String imageUrl =
      fileStorageService.store(image);
50 +              pet.setImageUrl(imageUrl);
51 +          }
52 +
53 +          Pet savedPet = petRepository.save(pet);
54 +          return toDto(savedPet);
55 +      }
56 +
57 +      /**
58 +       * Converte uma entidade {@link Pet} para seu
      respectivo {@link PetDto}.
59 +       */
60 +      private PetDto toDto(Pet pet) {
61 +          if (pet == null) {
62 +              return null;
63 +          }
64 +          PetDto dto = new PetDto();
65 +          BeanUtils.copyProperties(pet, dto);
66 +          if (pet.getTutor() != null) {
67 +              // No DTO, nós definimos apenas o ID
      do tutor.
68 +              dto.setTutorId(pet.getTutor().getId());
69 +          }
70 +          return dto;
71 +      }
72 +
73 +      /**
74 +       * Converte um {@link PetDto} para sua
      respectiva entidade {@link Pet}.
75 +       */
76 +      private Pet toEntity(PetDto dto) {
77 +          if (dto == null) {
78 +              return null;
79 +          }
80 +          Pet pet = new Pet();
81 +          //
82 +          // Copia as propriedades, mas ignora o
      tutorId, pois ele será tratado separadamente.
83 +          BeanUtils.copyProperties(dto, pet,
      "tutorId");
84 +          return pet;
85 +      }
86 +  }

```

```

5      import
      com.petconnectbe.repositories.UserRepository;
6      import com.petconnectbe.services.AddressService;
7      import com.petconnectbe.services.UserService;

8      import lombok.RequiredArgsConstructor;

9      import org.springframework.stereotype.Service;

10
11     import java.util.List;

12     import java.util.UUID;
13     import java.util.stream.Collectors;
14

15     @Service
16     @RequiredArgsConstructor
17     public class UserServiceImpl implements
      UserService {

18
19         private final UserRepository userRepository;
20         private final AddressService addressService;
21

22         @Override

23         public UserDto save(UserDto userDto) {
24             - if
      (userRepository.findByEmail(userDto.getEmail()).isPresent()) {
25             - throw new RuntimeException("Email já
      cadastrado.");
26             - }
27
28             - User user = new User();
29             - user.setName(userDto.getName());
30             - user.setType(userDto.getType());
31             - user.setEmail(userDto.getEmail());
32             - user.setPhone(userDto.getPhone());
33             -
      user.setBirthOrFoundationDate(userDto.getBirthOrFoundationDate());
34             -
      user.setCpfOrCnpj(userDto.getCpfOrCnpj());
35             - user.setPassword(userDto.getPassword());
36
37             - if (userDto.getEndereco() != null) {

```

```

5      import
      com.petconnectbe.repositories.UserRepository;
6      import com.petconnectbe.services.AddressService;
7      import com.petconnectbe.services.UserService;
8      + import
      jakarta.persistence.EntityNotFoundException;
9      import lombok.RequiredArgsConstructor;
10     + import org.springframework.beans.BeanUtils;
11     import org.springframework.stereotype.Service;
12     + import
      org.springframework.transaction.annotation.Transactional;

13
14     import java.util.List;
15     + import java.util.Optional;
16     import java.util.UUID;
17     import java.util.stream.Collectors;
18
19     + /**
20     +  * Implementação do serviço {@link UserService}
      para gerenciar os dados dos Usuários.
21     +  * <p>
22     +  * Esta classe gerencia a lógica de negócio para
      as operações de CRUD de usuários,
23     +  * validando dados, tratando exceções e
      interagindo com a camada de persistência.
24     +  */
25     @Service
26     @RequiredArgsConstructor
27     public class UserServiceImpl implements
      UserService {

28
29         private final UserRepository userRepository;
30         private final AddressService addressService;
31

32         /**
33         +  * {@inheritDoc}
34         +  *
35         +  * @throws IllegalArgumentException se o
      email fornecido já existir no sistema.
36         +  */
37         @Override
38         + @Transactional
39         public UserDto save(UserDto userDto) {
40         +
      userRepository.findByEmail(userDto.getEmail()).ifPresent(user -> {
41         +         throw new
      IllegalArgumentException("Email já cadastrado: "
      + userDto.getEmail());
42         +         });
43
44         +         User user = toEntity(userDto);

45
46         +         User savedUser =
      userRepository.save(user);

```

```

38 - user.setAddress(addressService.toEntity(userDto.getEndereco()));
39 -     }
40 -
41 -     User salvo = userRepository.save(user);
42 -     return toDto(salvo);
43 - }
44
45 @Override
46 - public UserDto findById(UUID id) {
47 -     User user = userRepository.findById(id)
48 -         .orElseThrow(() -> new
49 -             RuntimeException("Usuário não encontrado"));
49 -     return toDto(user);
50 - }
51
52 @Override
53 public List<UserDto> findAll() {
54     return userRepository.findAll()
55         .stream()
56         .map(this::toDto)
57         .collect(Collectors.toList());
58 }
59
60 @Override
61 public UserDto update(UUID id, UserDto
62     userDto) {
63 -     User user = userRepository.findById(id)
64 -         .orElseThrow(() -> new
65 -             RuntimeException("Usuário não encontrado para
66 -                 atualização."));
67 -
68 -     user.setName(userDto.getName());
69 -     user.setType(userDto.getType());
70 -
71 -     user.setEmail(userDto.getEmail());
72 -     user.setPhone(userDto.getPhone());
73 -
74 -     user.setBirthOrFoundationDate(userDto.getBirthOrF
75 -         oundationDate());
76 -
77 -     user.setCpfOrCnpj(userDto.getCpfOrCnpj());
78 -
79 -     if (userDto.getPassword() != null &&
80 -         !userDto.getPassword().isEmpty()) {
81 -
82 -         user.setPassword(userDto.getPassword());
83 -         return toDto(updatedUser);
84 -     }
85

```

```

47 +     return toDto(savedUser);
48
49
50 + /**
51 +  * {@inheritDoc}
52 +  */
53 @Override
54 + @Transactional(readOnly = true)
55 + public Optional<UserDto> findById(UUID id) {
56 +     return
57         userRepository.findById(id).map(this::toDto);
58
59 }
60
61 + /**
62 +  * {@inheritDoc}
63 +  */
64 @Override
65 + @Transactional(readOnly = true)
66 + public List<UserDto> findAll() {
67 +     return userRepository.findAll()
68 +         .stream()
69 +         .map(this::toDto)
70 +         .collect(Collectors.toList());
71
72 }
73
74 + /**
75 +  * {@inheritDoc}
76 +  *
77 +  * @throws EntityNotFoundException se nenhum
78 +  * usuário for encontrado com o ID fornecido.
79 +  */
80 @Override
81 + @Transactional
82 + public UserDto update(UUID id, UserDto
83     userDto) {
84 +     User user = userRepository.findById(id)
85 +         .orElseThrow(() -> new
86 +             EntityNotFoundException("Usuário não encontrado
87 +                 com o ID: " + id));
88
89 +     // Atualiza os campos do usuário
90 +     BeanUtils.copyProperties(userDto, user,
91 +         "id", "password", "email");
92
93 +     if (userDto.getPassword() != null &&
94 +         !userDto.getPassword().isEmpty()) {
95
96 +         user.setPassword(userDto.getPassword());
97 +         return toDto(updatedUser);
98
99 }
100
101 + /**
102 +  * {@inheritDoc}

```

```

86      @Override
87      public void deleteById(UUID id) {
88          if (!userRepository.existsById(id)) {
89 -             throw new RuntimeException("Usuário
não encontrado para exclusão.");
90          }
91          userRepository.deleteById(id);
92      }
93
94      private UserDto toDto(User user) {
95 -         UserDto userDto = new UserDto();
96 -         userDto.setUserId(user.getUserId());
97 -         userDto.setName(user.getName());
98 -         userDto.setType(user.getType());
99 -         userDto.setEmail(user.getEmail());
100 -         userDto.setPhone(user.getPhone());
101 -
102         userDto.setBirthOrFoundationDate(user.getBirthOrF
oundationDate());
103 -
104         userDto.setCpfOrCnpj(user.getCpfOrCnpj());
105 -         userDto.setPassword(null);
106
107         if (user.getAddress() != null) {
108 -
109         userDto.setEndereco(addressService.toDto(user.get
Address()));
110
111     }
112
113     return userDto;
114 }

```

```

101 +      *
102 +      * @throws EntityNotFoundException se nenhum
usuário for encontrado com o ID fornecido.
103 +      */
104 +      @Override
105 +      @Transactional
106 +      public void deleteById(UUID id) {
107 +          if (!userRepository.existsById(id)) {
108 +             throw new
EntityNotFoundException("Usuário não encontrado
com o ID: " + id);
109 +         }
110 +         userRepository.deleteById(id);
111 +     }
112
113     private UserDto toDto(User user) {
114 +         if (user == null) {
115 +             return null;
116 +         }
117 +         UserDto dto = new UserDto();
118 +         BeanUtils.copyProperties(user, dto,
"password"); // Nunca expor a senha
119
120         if (user.getAddress() != null) {
121 +
122         dto.setEndereco(addressService.toDto(user.getAddr
ess()));
123 +         }
124 +         return dto;
125 +     }
126 +     private User toEntity(UserDto dto) {
127 +         if (dto == null) {
128 +             return null;
129 +         }
130 +         User user = new User();
131 +         BeanUtils.copyProperties(dto, user);
132 +
133 +         if (dto.getEndereco() != null) {
134 +
135         user.setAddress(addressService.toEntity(dto.getEn
dereco()));
136 +         }
137 +         return user;
138 +     }

```

11 src/main/resources/application.yml

```

2      datasource:
3          # A linha abaixo diz ao Spring para usar o
driver do PostgreSQL
4          driver-class-name: org.postgresql.Driver
5 -         # O endereço do nosso banco de dados no Docker
6
7 -         url: jdbc:postgresql://db:5432/pet_connect
8 -         # As credenciais que definimos no docker-
compose.yml
9 -         username: postgres
10 -         password: postgres

```

```

2      datasource:
3          # A linha abaixo diz ao Spring para usar o
driver do PostgreSQL
4          driver-class-name: org.postgresql.Driver
5 +         # O endereço, usuário e senha do banco de
dados agora são lidos das variáveis de ambiente
6 +         url: ${DB_URL}
7 +         username: ${DB_USERNAME}
8 +         password: ${DB_PASSWORD}

```

```
10 jpa:
11 hibernate:
12     ddl-auto: update # Cria/atualiza as tabelas
    automaticamente
36 # h2:
37 #     console:
38 #         enabled: true
39 - #         path: /h2-console
```



```
9 jpa:
10 hibernate:
11     ddl-auto: update # Cria/atualiza as tabelas
    automaticamente
35 # h2:
36 #     console:
37 #         enabled: true
38 + #         path: /h2-console
```

✓ 21  src/test/resources/application.properties

```
... @@ -0,0 +1,21 @@
```

```
1 + # Configuração do Banco de Dados para Testes (H2
    em memória)
2 +
3 + # URL de conexão para o banco de dados H2 em
    memória.
4 + # "testdb" é o nome do banco de dados que será
    criado na memória.
5 + spring.datasource.url=jdbc:h2:mem:testdb
6 +
7 + # Driver JDBC para o banco de dados H2.
8 + spring.datasource.driverClassName=org.h2.Driver
9 +
10 + # Nome de usuário para o banco de dados H2.
11 + spring.datasource.username=sa
12 +
13 + # Senha para o banco de dados H2 (pode ser deixada
    em branco).
14 + spring.datasource.password=
15 +
16 + # Dialeto do Hibernate para o H2, para que o JPA
    saiba como se comunicar com o banco.
17 + spring.jpa.database-
    platform=org.hibernate.dialect.H2Dialect
18 +
19 + # Ação a ser tomada no banco de dados ao iniciar a
    aplicação.
20 + # "create-drop" cria o schema no início e o
    destrói no final. Perfeito para testes.
21 + spring.jpa.hibernate.ddl-auto=create-drop
```